Cartman PH — Mobile Applications Product Backlog

Tech Lead Note: This backlog covers the mobile development scope only (Rider App). All web applications (Admin Dashboard, Merchant Panel, Financial Ledger) are excluded from this tracking and will be handled separately.

1. Rider Logistics App (Core Scope)

Target Stack: Flutter or React Native, Background Telemetry Tracking, Supabase Realtime, and Firebase Cloud Messaging (FCM).

Epic: Order Acquisition Loop

Task R-1.1: Real-time Order Broadcast Feed

- **User Story:** As a Rider, I want to see a live pool of available delivery requests in my area the moment a merchant marks an order as ready, so that I can claim jobs quickly.
- **Technical Context:** Establish a **Supabase Realtime WebSockets** stream filtered to listen for orders where the state updates to READY_FOR_PICKUP.
- **Acceptance Criteria:**
    - New available orders append to the rider's feed instantly without manual swipe-to-refresh.

Task R-1.2: Order Acceptance Race-Condition Handling

- **User Story:** As a Rider, I want to tap an "Accept Order" button to claim a delivery, with the assurance that two riders cannot accidentally claim the same order.
- **Technical Context:** Implement an order acceptance loop using database-level transaction processing safety. When a rider accepts, it updates the orders table row with the assigned rider’s foreign key, requiring queries to execute under 50ms.
- **Acceptance Criteria:**
    - The first rider to execute the update secures the order; subsequent requests are rejected gracefully by database rules.

Epic: Active Logistics & Telemetry Tracking

Task R-2.1: Background Location Telemetry Stream

- **User Story:** As a Rider, I want the app to securely stream my GPS coordinates in the background while I am driving, so that dispatch and the customer can view my live location.
- **Technical Context:** Configure the mobile **geolocator** tool to track and stream background location coordinate steps. Optimize updates to preserve device battery life while on transit.

- **Acceptance Criteria:**
    - Coordinates are written continuously to the database background tracking logs at specified intervals.

Epic: Financial Integrity & Risk Management

Task R-3.1: Immutable Append-Only Wallet Ledger Display

- **User Story:** As a Rider, I want to view a detailed breakdown of my cash-on-hand liability and earnings history, so that I know exactly how much cash I owe the platform.
- **Technical Context:** Bind this UI view to the rider_wallet_transactions table. **CRUCIAL:** Do not allow the mobile app to pass arbitrary balance modifications. The wallet balance must be an aggregate representation calculated strictly via the mathematical model:
  $$W_{r} = \Sigma R_{a} - \Sigma (V_{o(COD)} + (V_{o} \times C_{m}) - F_{d})$$
  Where W_r is net disposable cash, R_a is cash remittances, V_o is historical order value, C_m is commission splits, and F_d is delivery payouts.
- **Acceptance Criteria:**
    - Rider can review an audit trail of credit_remittance, debit_cod_order, and credit_delivery_reward transaction types.

Task R-3.2: Automated Negative Balance Lockout Core

- **User Story:** As a Rider, I understand that if my cash-on-hand liability exceeds safe limits, the app will suspend my account so that I am prompted to remit funds to management.
- **Technical Context:** Read the aggregated balance condition at the database layer. If the computed net wallet position drops below the threshold limit ($W_{r} \le -P2,000$), trigger a hard client lockout.
- **Acceptance Criteria:**
    - If the threshold is breached, the app interface must automatically hide the available order feed and display a restriction screen blocking order acceptance loops until an admin credit transaction is logged.

Epic: Dispatch Coordination

Task R-4.1: Accept / Decline Order Engine

- **User Story:** As a Rider, when a delivery request pops up on my feed, I want to see its pickup location, drop-off location, and payout, with the option to accept or decline it within a limited window.
- **Technical Context:** Use individual row-state variables to animate incoming requests. Declining an order hides it locally from the rider's active view array (stored in Hive/AsyncStorage local memory).
- **Acceptance Criteria:**
    - Tapping "Accept" runs the Supabase relational write check to claim the order.
    - Tapping "Decline" clears the request card safely from the current screen view.

Task R-4.2: Rider Availability Toggle (On-Duty / Off-Duty)

- **User Story:** As a Rider, I want to flip a switch to go "On-Duty" when I start working or "Off-Duty" when I take a break, so that the system knows whether to send me orders.
- **Technical Context:** Update an is_active boolean flag inside the riders table. When false, the database Row-Level Security (RLS) or system filters exclude this rider from receiving real-time broadcast pushes.
- **Acceptance Criteria:**
    - A prominent toggle switch on the home dashboard.
    - Going "Off-Duty" successfully disconnects active real-time location streaming to preserve phone battery.

Task R-4.3: Native Navigation Mapping Integration

- **User Story:** As a Rider, I want to tap a navigation button on an active order to open external turn-by-turn routing, so that I can find the merchant or customer quickly.
- **Technical Context:** Implement deep-linking schemes to pass the geocoded drop-off or merchant coordinates directly to native Google Maps, Apple Maps, or Waze applications installed on the rider's device.
- **Acceptance Criteria:**
    - Tapping "Navigate" successfully launches the device's default mapping application with the precise target coordinates pre-filled.

Epic: Order Execution & Progression

Task R-5.1: Delivery Status Update Steps

- **User Story:** As a Rider, I want to manually update the order status via simple button actions as I complete each milestone, so that the entire operational system updates simultaneously.
- **Technical Context:** Provide a single contextual action button that mutates the order status row sequentially: ACCEPTED $\rightarrow$ ARRIVED_AT_MERCHANT $\rightarrow$ PICKED_UP $\rightarrow$ DELIVERED.
- **Acceptance Criteria:**
    - Each button tap triggers an immediate database write updating the order status column, which broadcasts seamlessly to web dashboards and customer apps.

Epic: Earnings, Audits & Analytics

Task R-6.1: Real-Time Earnings Monitoring

- **User Story:** As a Rider, I want to see my total accumulated earnings and rewards for the day or week directly on my dashboard, so that I can track my performance.
- **Technical Context:** Aggregate the credit_delivery_reward rows for the current day from the rider_wallet_transactions ledger.
- **Acceptance Criteria:**
    - Displays a clean numerical text block showing "Today's Take-Home Earnings: ₱X.XX".

Task R-6.2: Rider Activity Logs & Order History

- **User Story:** As a Rider, I want to see a log of all the specific deliveries I have completed, including distances and individual payouts, to cross-reference my physical work.
- **Technical Context:** Fetch historic rows from the orders table where assigned_rider_id matches the user and state equals delivered.
- **Acceptance Criteria:**
    - Rider can review a list showing: Order Reference Number, Timestamp, Delivery Payout Amount (F_d), and Cash Collected Status (COD).

Task R-6.3: Basic Rider Performance Metrics

- **User Story:** As a Rider, I want to see basic summary metrics like my overall Acceptance Rate and average delivery completion speed, so that I know if I'm meeting platform standards.
- **Technical Context:** Compute basic telemetry stats from historical order logs (e.g., mathematically dividing total accepted requests by total broadcasted instances to that user ID).
- **Acceptance Criteria:**
    - A dedicated "Performance" tab rendering a scannable summary layout containing: Acceptance Rate %, Total Completed Trips, and Average Delivery Duration.